

MVP – Minimum Viable Product

1. Objectif du document

Ce document définit le **MVP (Minimum Viable Product)** de l'application de gestion des flux financiers.

Il a pour but de : - Clarifier ce qui sera **développé en priorité** - Aligner les parties prenantes (produit, technique, business) - Éviter le sur-développement prématuré - Servir de référence avant le démarrage de la phase de développement

Le MVP représente la **plus petite version fonctionnelle** de l'application apportant une **valeur réelle et mesurable** aux utilisateurs.

2. Définition du MVP

Un MVP n'est **pas une version incomplète** ni une version de mauvaise qualité.

C'est une version : - Fonctionnelle - Sécurisée - Stable - Axée sur la valeur métier principale

Le MVP valide le problème à résoudre, pas toutes les fonctionnalités possibles.

3. Vision du MVP

Problématique utilisateur

« Je gagne de l'argent, je dépense de l'argent, mais je ne sais pas exactement où va mon argent ni quelle est ma situation financière réelle. »

Proposition de valeur

Donner à l'utilisateur une vision claire, simple et fiable de ses flux financiers.

4. Cible du MVP

Le MVP s'adresse à un public large : - Salariés - Freelancers - Étudiants - Auto-entrepreneurs

Aucun prérequis financier ou comptable n'est nécessaire.

5. Périmètre fonctionnel du MVP

5.1 Sécurité et authentification (fondation)

- Authentification via :
 - Google
 - Facebook
 - Inscription locale
- Gestion des identités via **Keycloak**
- Authentification forte obligatoire (MFA) :
 - OTP basé sur TOTP
 - Utilisation d'une application Authenticator

La sécurité n'est pas optionnelle, même dans un MVP.

5.2 Gestion du profil utilisateur

- Création automatique du profil après authentification
 - Informations de base :
 - Nom
 - Email
 - Devise principale
 - Langue
-

5.3 Gestion des comptes financiers

- Création de comptes financiers :
 - Compte courant
 - Cash
 - Carte bancaire
 - Solde initial
 - Devise par compte
-

5.4 Gestion des flux financiers (cœur du MVP)

Flux entrants

- Salaire
- Revenus freelances
- Autres revenus

Flux sortants

- Loyer

- Nourriture
- Transport
- Loisirs
- Autres dépenses

Fonctionnalités : - Ajouter une transaction - Modifier une transaction - Supprimer une transaction - Associer une catégorie - Associer un compte

5.5 Catégories

- Catégories par défaut
 - Création de catégories personnalisées
 - Une transaction appartient à une seule catégorie
-

5.6 Consultation et historique

- Liste des transactions
 - Pagination
 - Filtres simples :
 - Période
 - Catégorie
 - Type (entrant / sortant)
-

5.7 Dashboard (valeur immédiate)

- Total des revenus (par période)
 - Total des dépenses (par période)
 - Solde global
 - Répartition par catégorie (graphique simple)
 - Vue mensuelle
-

6. Fonctionnalités exclues du MVP

Les fonctionnalités suivantes sont volontairement exclues du MVP :

- Budgets avancés
- Prédictions financières / IA
- Connexion bancaire (Open Banking)
- Gestion des investissements et actions
- Exports (PDF, Excel)
- Notifications avancées

- Partage de comptes
- Multi-utilisateurs par compte

Ces fonctionnalités seront étudiées après validation du MVP.

7. Contraintes techniques du MVP

- Backend : Spring Boot
- Architecture : Hexagonale (Ports & Adapters)
- Sécurité : OAuth2 / OIDC via Keycloak
- Frontend : Angular
- Base de données : PostgreSQL

Même dans le MVP : - Code propre - Architecture respectée - Tests unitaires sur le cœur métier

8. Indicateurs de succès du MVP

Le MVP est considéré comme validé si : - L'utilisateur comprend sa situation financière en quelques minutes - L'utilisateur utilise régulièrement le dashboard - Les transactions sont ajoutées de manière récurrente - Le feedback utilisateur est globalement positif

9. Évolution post-MVP

Après validation du MVP : - Enrichissement fonctionnel - Ajout de fonctionnalités premium - Optimisations UX et performance - Intégration de services externes

10. Conclusion

Le MVP constitue la **fondation produit** de l'application.

Il permet de livrer rapidement une valeur mesurable, tout en posant une architecture robuste et évolutive pour les futures versions.

Auteur : Yassin MELLOUKI

Rôle : Product & Backend Architect

Date : 2026-02-07